Unit 5 - Coding symbolic data

Essential reading

Wayner, Peter *Compression Algorithms for Real Programmers*. (Morgan Kaufmann, 2000) [ISBN 0-12-788774-1]. Chapter 2.3-2.5.

Further reading

Sayood, Khalid *Introduction to Data Compression* (Morgan Kaufmann, 1996) [ISBN 1-55860-346-8]. Chapter 2.

In this chapter, we shall look more closely at the structure of compression algorithms in general. Starting with symbolic data compression, we apply the information theory to gain a better understanding of compression algorithms. Some conclusions we draw from this chapter may also be useful for multimedia data compression in later chapters.

Compression algorithms

You will recall that in the Introduction, we said that data compression essentially consists of two types of work: modelling and coding. It is often useful to consciously consider the two entities of compression algorithms separately.

- The general model is the embodiment of what the compression algorithm knows about the source domain. Every compression algorithm has to make use of some knowledge about its platform.

  Example 4.1 *Consider the Huffman encoding algorithm. The model is based on the probability distribution of characters of a source text.*
- The device that is used to fulfil the task of coding is usually called *coder* meaning *encoder*. Based on the model and some calculations, the coder is used to
  - derive a code
  - encode (compress) the input.

  Example 4.2 *Consider the Huffman encoding algorithm again. The coder assigns shorter codes to the more frequent symbols and longer codes to infrequent ones.*

A similar structure applies to decoding algorithms. There is again a *model* and a *decoder* for any decoding algorithm.

Conceptually, we can distinguish *two* types of compression algorithms, namely, *static*, or *adaptive* compression, based on whether the model structure may be updated during the process of compression or decompression.

The model-coder structures can be seen clearly in diagrams Figure 4.1 and Figure 4.2:

- **Static (non-adaptive) system** (Figure 4.1): This model remains unchanged during the compression or decompression process.

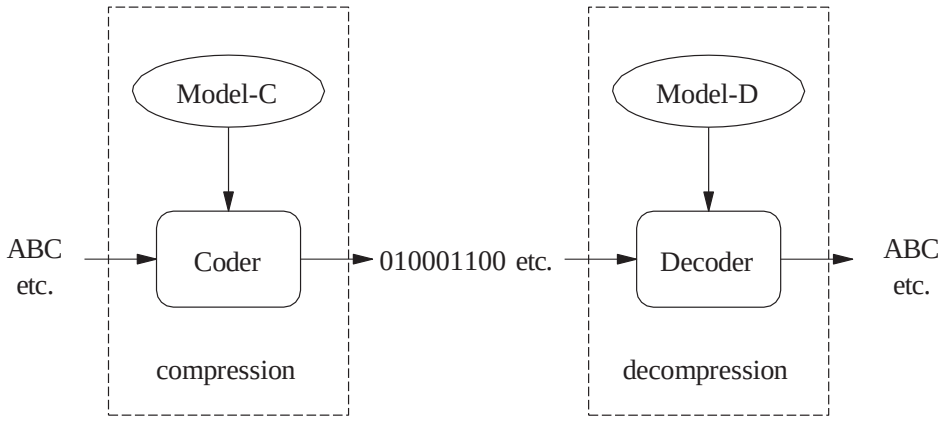

Figure 4.1: A static or non-adaptive compression system

- **Adaptive system** (Figure 4.2): The model may be changed during the compression or decompression process according to the change of input (or feedback from the output).
  Some adaptive algorithms actually build the model based on the input starting from an empty model.

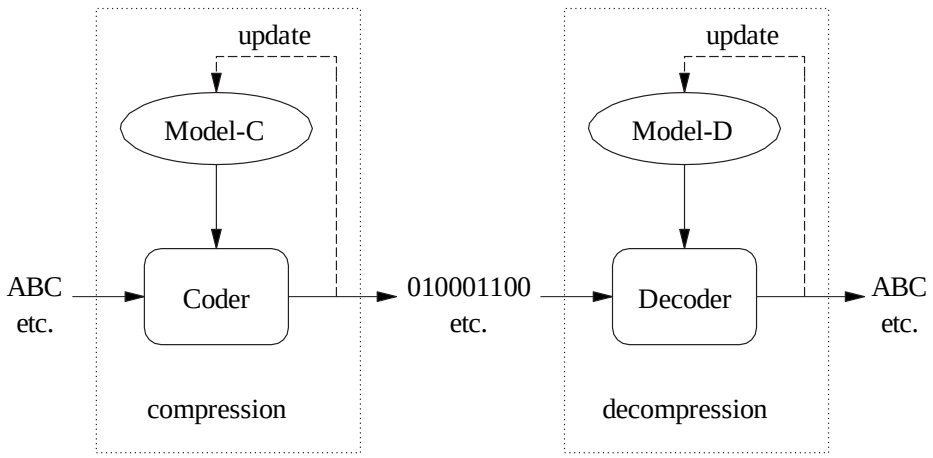

Figure 4.2: Adaptive compression system

In practice, the software or hardware for implementing the model is often a mixture of static and adaptive algorithms, for various reasons such as efficiency.

Symmetric and asymmetric compression

In some compression systems, the model for compression (Model-C in the figures) and that for decompression (Model-D) are identical. If they are identical, the compression system is called *symmetric*, otherwise, it is said to be *non-symmetric*. The compression using a symmetric system is called *symmetric compression*, and the compression using an asymmetric system is called *asymmetric compression*.

Coding methods

In terms of the length of codewords used *before* or *after* compression, compression algorithms can be classified into the following categories:

1. **Fixed-to-fixed**: each symbol before compression is represented by a fixed number of bits (e.g. 8 bits in ASCII format) and is encoded as a sequence of bits of a fixed length after compression.

   Example 4.3 *A:00, B:01, C:10, D:11* [1]

2. **Fixed-to-variable**: each symbol before compression is represented by a fixed number of bits and is encoded as a sequence of bits of different length.

   Example 4.4 *A:0; B:10; C:101; D:0101.*

3. **Variable-to-fixed**: a sequence of symbols represented in different number of bits before compression is encoded as a fixed-length sequence of bits.

   Example 4.5 *ABCD:00; ABCDE:01; BC:11.*

4. **Variable-to-variable**: a sequence of symbols represented in different number of bits before compression is encoded as a variable-length sequence of bits.

   Example 4.6 *ABCD:0; ABCDE:01; BC:1; BBB:0001.*

[1] *For ease of read, the symbol itself is used here instead of its ASCII codeword.*

Question 4.1 *Which class does Huffman coding belong to?*

Solution It belongs to the *fixed-to-variable* class. Why? Each symbol before compression is represented by a fixed length code, e.g. 8 bits in ASCII, and the codeword for each symbol after compression consists of different number of bits.

Question of unique decodability

The issue of unique decodability arises during the decompression process when a variable length code is used. Ideally, there is only one way to decode a sequence of bits consisting of codewords. However, when symbols are encoded by a variable-length code, there may be more than one way to identifying the codewords from the sequence of bits.

Given a variable length code and a sequence of bits to decompress, the code is regarded as *uniquely decodable* if there is only one possible way to decode the bit sequence in terms of the codewords.

Example 4.7 *Given symbols A, B, C and D, we wish to encode them as follows: A:0; B:10; C:101; D:0101. Is this code uniquely decodable?*

The answer is 'No'. Because an input such as '0101101010' can be decoded in more than one way, for example, as ACCAB or as DCAB.

However, for the example above, there is a solution if we introduce a new symbol to separate each codeword. For example, if we use a 'stop' symbol "/". We could then encode DDCAB as '0101/0101/101/0/10'. At the decoding end, the sequence '0101/0101/101/0/10' will be easily decoded uniquely.

Unfortunately, the method above is too costly because of the extra symbol "/" introduced. Is there any alternative approach which allows us to uniquely decode a compressed message using a code with various length codewords? After all, how would we know whether a code with various length codewords is uniquely decodable?

Well, one simple solution is to find another code which is a so-called *prefix code* such as (0,11,101,1001) for (A,B,C,D).

Prefix and dangling suffix

Let us look at some concepts first:

- Prefix: Consider two binary codewords w_1 and w_2 with lengths k and n bits respectively, where $k < n$. If the first k bits of w_2 are identical to w_1, then w_1 is called a *prefix* of w_2.
- Dangling Suffix: The remain of last $n - k$ bits of w_2 is called the *dangling suffix*.

Example 4.8 *Suppose* w_1 = 010, w_2 = <u>010</u>11. *Then the prefix of* w_2 *is 010 and the suffix is 11.*

Prefix codes

A prefix code is a code in which *no* codeword is a prefix to another codeword (Note: meaning 'prefix-free codes').
This occurs when no codeword for one symbol is a prefix of the codeword for another symbol.

Example 4.9 *The code (1, 01, 001, 0000) is a prefix code since no codeword is a prefix of another codeword.*

The code (0, 10, 110, 1011) is not a prefix code since 10 is a prefix of 1011.

Prefix codes are important in terms of uniquely decodability for the following two main reasons (See Sayood(2000), section 2.4.3 for the proof).

1. Prefix codes are uniquely decodable.

   This can be seen from the following informal reasoning:

   Example 4.10 *Draw a 0-1 tree for each code above, and you will see the difference. For a prefix code, the codewords are only associated with the leaves.*

```
          root                          root
       0/      \1                    0/      \1
      0/ \1     A                  A        0/   \1
   0/  \1    B                          (B)    0/
  0/     C                              \1      C
  D                                       \1
                                           D
```

2. For any non-prefix code whose codeword lengths satisfy certain condition (see section 'Kraft-McMillan inequality' below), we can always find a prefix code with the same codeword length distribution.

   Example 4.11 *Consider code (0, 10, 110, 1011).*

Kraft-McMillan inequality

Theorem 4.1 *Let C be a code with N codewords with lengths $l_1, l_2, \cdots, l_N$. If C is uniquely decodable, then*

$$K(C) = \sum_{i=1}^{N} 2^{-l_i} \leq 1$$

This inequality is known as the Kraft-McMillan inequality. (See Sayood(2000), section 2.4.3 for the proof).

In $\sum_{i=1}^{N} 2^{-l_i} \leq 1$, N is the number of codewords in a code, l_i is the length of the ith codeword.

Example 4.12 *Given an alphabet of 4 symbols (A, B, C, D), would it be possible to find a uniquely decodable code in which a codeword of length 2 is assigned to A, length 1 to B and C, and length 3 to D?*

Solution Here we have $l_1 = 2$, $l_2 = l_3 = 1$, and $l_4 = 3$.

$$\sum_{i=1}^{4} 2^{-l_i} = \frac{1}{2^2} + \frac{1}{2} + \frac{1}{2} + \frac{1}{2^3} > 1$$

Therefore, we cannot hope to find a uniquely decodable code in which the codewords are of these lengths.

Example 4.13 *If a code is a prefix code, what can we conclude about the lengths of the codewords?*

Solution Since prefix codes are uniquely decodable, they must satisfy the Kraft-McMillan Inequality.

Some information theory

The information theory is based on mathematical concepts of probability theory. The term *information* carries a sense of unpredictability in transmitted messages. The *information source* can be represented by a set of event symbols (random variables) from which the information of each event can be measured by the surprise that the event may cause, and by the probability rules that govern the emission of these symbols.

The symbol set is frequently called the *source alphabet*, or *alphabet* for short. The number of elements in the set is called *cardinality* ($|A|$).

Self-information

This is defined by the following mathematical formula:

$$I(A) = -\log_b P(A),$$

where A is an event, $P(A)$ is the probability that event A occurs.

The logarithm base (i.e. b in the formula) may be in:

- unit *bits*: base 2 (used in the subject guide)
- unit *nats*: base e
- unit *hartleys*: base 10.

The self-information of an event measures the amount of one's surprise evoked by the event. The negative logarithm $-\log_b P(A)$ can be written as

$$\log_b \frac{1}{P(A)}.$$

Note that $\log(1) = 0$, and that $|-\log(P(A))|$ increases as $P(A)$ decreases from 1 to 0. This supports our intuition from daily experience. For example, a low-probability event tends to cause more surprise.

Entropy

The precise mathematical definition of this measure was given by Shannon. It is called *entropy of the source* which is associated with the experiments on the (random) event set.

$$H = \sum P(A_i)I(A_i) = -\sum P(A_i)\log_b P(A_i),$$

where source $|A| = (A_1, \ldots, A_N)$.

The information content of a source is an important attribute. Entropy describes the average amount of information converged per source symbol. This can also be thought to measure the expected amount of surprise caused by the event.

If the experiment is to take out the symbols A_i from a source A, then

- the entropy is a measure of the minimum average number of binary symbols (bits) needed to encode the output of the source.
- Shannon showed that the best that a lossless symbolic compression scheme can do is to encode the output of a source with an average number of bits equal to the entropy of the source.

Example 4.14 *Consider the three questions below:*

1. *Given four symbols A, B, C and D, the symbols occur with an equal probability. What is the entropy of the distribution?*
2. *Suppose they occur with probabilities 0.5, 0.25, 0.125 and 0.125 respectively. What is the entropy associated with the event (experiment)?*
3. *Suppose the probabilities are 1,0,0,0. What is the entropy?*

Solution

1. The entropy is $1/4(-\log_2(1/4)) \times 4 = 2$ bits

2. The entropy is $0.5 * 1 + 0.25 * 2 + 0.125 * 3 + 0.125 * 3 = 1.75$ bits
3. The entropy is 0 bit.

Optimum codes

In information theory, the ratio of the entropy of a source to the average number of binary bits used to represent the source data is a measurement of the information *efficiency* of the source. In data compression, the ratio of the entropy of a source to the average length of the codewords of a code can be used to measure how successful the code is for compression.

Here a source is usually described as an alphabet $\alpha = \{s_1, \cdots, s_n\}$ and the next symbol chosen randomly from α is s_i with probability $Pr[s_i] = p_i$, $\sum_{i=1}^{n} p_i = 1$.

Information Theory says that *the best that a lossless symbolic compression scheme can do is to encode the output of a source with an average number of bits equal to the entropy of the source.*

We write done the entropy of the source:

$$\sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i} = p_1 \log_2 \frac{1}{p_1} + p_2 \log_2 \frac{1}{p_2} + \cdots + p_n \log_2 \frac{1}{p_n}$$

Using a variable length code to the symbols, l_i bits for s_i , the average number of bits is

$$\bar{l} = \sum_{i=1}^{n} l_i p_i = l_1 p_1 + l_2 p_2 + \cdots + l_n p_n$$

The code is optimum if the average length equals to the entropy. Let

$$\sum_{i=1}^{n} l_i p_i = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i}$$

and rewrite it as

$$\sum_{i=1}^{n} p_i l_i = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i}$$

This means that it is *optimal* to encode each s_i with $l_i = -\log_2 p_i$ bits, where $1 \le i \le n$.

It can be proved that:

1. the average length of any uniquely decodable code, e.g. prefix codes must be $\geq$ the entropy
2. the average length of a uniquely decodable code is equal to the entropy *only* when, for all i, $l_i = -log_2 p_i$, where l_i is the length and p_i is the probability of codeword$_i$[2].
3. the average codeword length of the Huffman code for a source is greater and equal to the entropy of the source and less than the entropy plus 1. [Sayood(2000), section 3.2.3]

[2] *This can only happen if all probabilities are negative powers of 2 in Huffman codes, for l_i has to be an integer (in bits).*

Scenario

What do the concepts such as *variable length codes*, *average length of codewords* and *entropy* mean in practice?

Let us see the following scenario: Ann and Bob are extremely good friends but they live far away to each other. They are both very poor students financially. Ann wants to send a shortest message to Bob in order to save her money. In fact, she decides to send to Bob a *single* symbol from their own secret alphabet. Suppose that:

- the next symbol that Ann wants to send is randomly chosen with a known probability
- Bob knows the alphabet and the probabilities associated with the symbols.

Ann knows that you have studied the Data compression so she asks you the important questions in the following example:

Example 4.15 *Consider the three questions as below:*

1. *To minimise the average number of bits Ann uses to communicate her symbol to Bob, should she assign a fixed length code or a variable length code to the symbols?*
2. *What is the average number of bits needed for Ann to communicate her symbol to Bob?*
3. *What is meant by a 0 entropy? For example, what is meant if the probabilities associated with the alphabet are {0,0,1, · · ·, 0}?*

You give Ann the following:

Solution

1. She should use variable length codes because she is likely to use some symbols more frequently than others. Using variable length codes can save bits hopefully.
2. Ann needs at least the average number of bits that equal to the entropy of the source. That is $-\sum_{i=1}^{n} p_i \log_2 p_i$ bits.
3. A '0 entropy' means that the minimum average number of bits that Ann needs to send to Bob is zero.

   Probability distribution {0,0,1, · · · , 0} means that Ann will definitely send the third symbol in the alphabet as the next symbol to Bob and Bob knows this.

   If Bob knows what Ann is going to say then she does not need to say anything, does she?!

Learning outcomes

On completion of your studies in this chapter, you should be able to:

- explain why modelling and coding are usually considered separately for compression algorithm design
- identify the model and the coder in a compression algorithm
- distinguish a static compression system by an adaptive one
- identify prefix codes
- demonstrate the relationship between prefix codes, Kraft-McMillan inequality and the uniquely decodability

explain